

BRAC'T's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 8

Roll no: 57 PRN No:12410559 Name of Student: Avni Selote

Subject Name : Deep Learning

Problem Statement : Text Classification using Pretrained BERT

Dataset :

This is the classification based E-commerce text dataset for 4 categories - "Electronics", "Household", "Books" and "Clothing & Accessories", which almost cover 80% of any E-commerce website. The dataset is in ".csv" format with two columns - the first column is the class name and the second one is the datapoint of that class. The data point is the product and description from the e-commerce website.

The dataset has the following features :

1. Data Set Characteristics: Multivariate
2. Number of Instances: 50425
3. Number of classes: 4
4. Area: Computer science
5. Attribute Characteristics: Real
6. Number of Attributes: 1
7. Associated Tasks: Classification
8. Missing Values? No

Theory :

BERT (Bidirectional Encoder Representations from Transformers) is a pretrained language model built on the Transformer encoder architecture. Unlike earlier models that read text left-to-right, BERT uses self-attention to look at a word's full left and right context simultaneously, producing deep *bidirectional* contextual embeddings. It is pretrained on massive text corpora using two unsupervised tasks: Masked Language Modeling (predicting randomly masked words) and Next Sentence Prediction (predicting whether two sentences follow each other). This pretraining gives BERT a strong general understanding of language, which we reuse via

transfer learning: instead of training a model from scratch, we take BERT's pretrained weights and fine-tune them on our own labeled dataset (product descriptions → categories) by attaching a small classification head on top of BERT's [CLS] token representation, which acts as a summary vector for the whole input sequence.

Algorithm:

1. Load the raw dataset and clean the text (lowercase, remove noise).
2. Encode category labels to integers and split data into train/validation/test sets.
3. Load the pretrained BERT tokenizer/preprocessor to convert raw text into token IDs the model understands.
4. Load the pretrained BERT model and attach a classification head (Dense layer with softmax over the categories).
5. Compile the model with an optimizer (Adam, small learning rate), loss (categorical crossentropy), and accuracy metric.
6. Fine-tune the model on the training set, monitoring validation loss/accuracy across a few epochs.
7. Evaluate the fine-tuned model on the held-out test set (accuracy, classification report, confusion matrix).
8. Use the trained model to predict categories for new, unseen product descriptions.

Output :

1) Importing required libraries

Text Classification using Pretrained BERT

E-commerce Product Description Classification

Dataset: [E-commerce Text Classification](#) (Kaggle)

1. Install & Import Required Libraries

```
!pip install -q kagglehub keras-hub tensorflow scikit-learn pandas matplotlib seaborn
```

```
# ---- Set the Keras 3 backend to TensorFlow BEFORE importing keras / keras_hub ----  
# (Keras 3 supports multiple backends: tensorflow, jax, torch. This assignment requires  
# TensorFlow/Keras only, so we pin the backend explicitly.)  
import os  
os.environ["KERAS_BACKEND"] = "tensorflow"  
  
# ---- Core libraries ----  
import re  
import glob  
import numpy as np  
import pandas as pd
```

```

# ---- Visualization ----
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Scikit-learn utilities ----
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix

# ---- TensorFlow / Keras (deep learning framework used in this notebook) ----
import tensorflow as tf
import keras

# ---- KerasHub: provides the pretrained BERT model + built-in tokenizer/preprocessor ----
import keras_hub

# Reproducibility
SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)
keras.utils.set_random_seed(SEED)

print('Keras backend in use:', keras.backend.backend())
print('TensorFlow version:', tf.__version__)
print('GPU available:', tf.config.list_physical_devices('GPU'))

```

```

... Keras backend in use: tensorflow
TensorFlow version: 2.20.0
GPU available: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

```

2) Downloading the dataset

```

import kagglehub

# Download latest version of the dataset from Kaggle
path = kagglehub.dataset_download("saurabhshahane/ecommerce-text-classification")

print("Path to dataset files:", path)

# List all files inside the downloaded dataset folder
for f in os.listdir(path):
    print("-", f)

```

```

... Using Colab cache for faster access to the 'ecommerce-text-classification' dataset.
Path to dataset files: /kaggle/input/ecommerce-text-classification
- ecommerceDataset.csv

```

3) Load and explore the dataset

```

# Automatically find the main CSV file inside the downloaded dataset folder
csv_files = glob.glob(os.path.join(path, "**", "*.csv"), recursive=True)
print("CSV files found:", csv_files)

# Pick the file that looks like the main dataset (largest CSV file, usually the correct one)
csv_path = max(csv_files, key=os.path.getsize)
print("\nUsing file:", csv_path)

# The original file has no header, so we assign column names manually:
# column 1 -> product category (label), column 2 -> product description (text)
df = pd.read_csv(csv_path, header=None, names=["category", "text"])

print("Dataset shape:", df.shape)
df.head()

```

```

... CSV files found: ['/kaggle/input/ecommerce-text-classification/ecommerceDataset.csv']

```

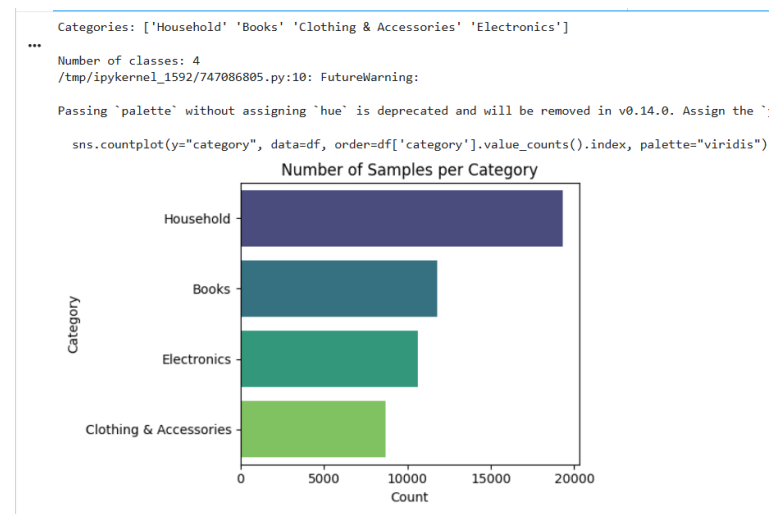
```

Using file: /kaggle/input/ecommerce-text-classification/ecommerceDataset.csv
Dataset shape: (50425, 2)

```

	category	text
0	Household	Paper Plane Design Framed Wall Hanging Motivat...
1	Household	SAF 'Floral' Framed Painting (Wood, 30 inch x ...
2	Household	SAF 'UV Textured Modern Art Print Framed' Pain...
3	Household	SAF Flower Print Framed Painting (Synthetic, 1...
4	Household	Incredible Gifts India Wooden Happy Birthday U...

4) Visualization of number of samples per category



5) Clean and preprocess the text

```
def clean_text(text):
    """Basic text cleaning: lowercase, remove extra spaces and non-alphanumeric noise."""
    text = str(text).lower()
    text = re.sub(r"[^a-z0-9\s]", " ", text) # remove special characters
    text = re.sub(r"\s+", " ", text).strip() # collapse multiple spaces
    return text

df["clean_text"] = df["text"].apply(clean_text)

# Remove any rows that became empty after cleaning
df = df[df["clean_text"].str.len() > 0].reset_index(drop=True)

df[["category", "text", "clean_text"]].head()
```

	category	text	clean_text
0	Household	Paper Plane Design Framed Wall Hanging Motiv...	paper plane design framed wall hanging motivat...
1	Household	SAF 'Floral' Framed Painting (Wood, 30 inch x ...	saf floral framed painting wood 30 inch x 10 i...
2	Household	SAF 'UV Textured Modern Art Print Framed' Pain...	saf uv textured modern art print framed painti...
3	Household	SAF Flower Print Framed Painting (Synthetic, 1...	saf flower print framed painting synthetic 13 ...
4	Household	Incredible Gifts India Wooden Happy Birthday U...	incredible gifts india wooden happy birthday u...

6) Encoding labels and train test split

```
# ---- Encode text labels into numbers (e.g. 'Electronics' -> 0, 'Books' -> 1, ...) ----
label_encoder = LabelEncoder()
df["label"] = label_encoder.fit_transform(df["category"])
num_classes = df["label"].nunique()

print("Classes:", list(label_encoder.classes_))
print("Number of classes:", num_classes)

# ---- Optional: subsample the dataset to keep training time reasonable ----
SAMPLE_SIZE = 6000 # set to None to use the FULL dataset (will take much longer to train)

if SAMPLE_SIZE is not None and SAMPLE_SIZE < len(df):
    df_sample = df.groupby("label", group_keys=False).apply(
        lambda x: x.sample(int(SAMPLE_SIZE / num_classes), random_state=SEED, replace=False)
    ).reset_index(drop=True)
else:
    df_sample = df.copy()

print("\nWorking dataset size:", df_sample.shape)

# ---- Train / Validation / Test split (70% / 15% / 15%) ----
train_texts, temp_texts, train_labels, temp_labels = train_test_split(
    df_sample["clean_text"].tolist(),
    df_sample["label"].tolist(),
    test_size=0.30,
    random_state=SEED,
    stratify=df_sample["label"],
)

val_texts, test_texts, val_labels, test_labels = train_test_split(
    temp_texts, temp_labels,
    test_size=0.50,
    random_state=SEED,
    stratify=temp_labels,
)

train_labels = np.array(train_labels)
val_labels = np.array(val_labels)
test_labels = np.array(test_labels)
```

7) BERT Classification Model

```
PRESET = "bert_base_en_uncased" # pretrained BERT checkpoint (uncased, base size)
MAX_LEN = 128 # max number of tokens per product description

# Load a pretrained BERT classifier: backbone + tokenizer/preprocessor + classification head,
# all bundled together as one Keras Model.
classifier = keras_hub.models.BertTextClassifier.from_preset(
    PRESET,
    num_classes=num_classes,
    activation="softmax", # output class probabilities directly
    preprocessor=keras_hub.models.BertTextClassifierPreprocessor.from_preset(
        PRESET, sequence_length=MAX_LEN
    ),
)

classifier.summary()
```

Downloading to /root/.cache/kagglehub/models/keras/bert/keras/bert_base_en_uncased/3/config.json...
 100%|██████████| 457/457 [00:00<00:00, 787kB/s]
 Downloading to /root/.cache/kagglehub/models/keras/bert/keras/bert_base_en_uncased/3/tokenizer.json...
 100%|██████████| 761/761 [00:00<00:00, 1.24MB/s]
 Downloading to /root/.cache/kagglehub/models/keras/bert/keras/bert_base_en_uncased/3/assets/tokenizer/vocabulary.txt...
 100%|██████████| 226k/226k [00:00<00:00, 1.10MB/s]
 Preprocessor: "bert_text_classifier_preprocessor"

Layer (type)	Config
bert_tokenizer (BertTokenizer)	Vocab size: 30,522

Model: "bert_text_classifier"

Layer (type)	Output Shape	Param #	Connected to
padding_mask (InputLayer)	(None, None)	0	-
segment_ids (InputLayer)	(None, None)	0	-
token_ids (InputLayer)	(None, None)	0	-
bert_backbone (BertBackbone)	[(None, 768), (None, None, 768)]	109,482,240	padding_mask[0][0], segment_ids[0][0], token_ids[0][0]
classifier_dropout (Dropout)	(None, 768)	0	bert_backbone[0][0]
logits (Dense)	(None, 4)	3,076	classifier_dropout[0][0]

Total params: 109,485,316 (417.65 MB)
 Trainable params: 109,485,316 (417.65 MB)
 Non-trainable params: 0 (0.00 B)

8) Compile and train the model

```

# Fine-tuning BERT requires a very small learning rate
classifier.compile(
    optimizer=keras.optimizers.Adam(learning_rate=2e-5),
    loss=keras.losses.SparseCategoricalCrossentropy(),
    metrics=["accuracy"],
)

early_stop = keras.callbacks.EarlyStopping(monitor="val_loss", patience=2, restore_best_weights=True)

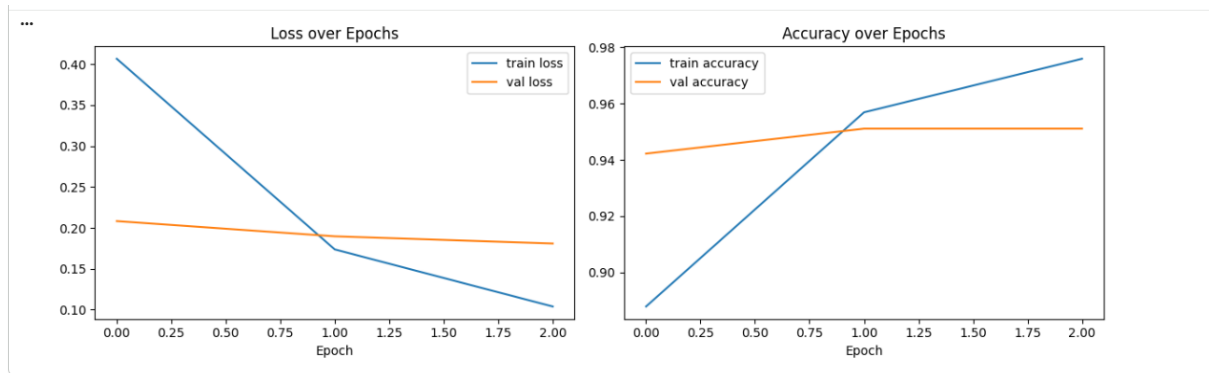
EPOCHS = 3          # small number of epochs is usually enough when fine-tuning BERT
BATCH_SIZE = 16

# Raw text strings can be passed directly as `x` -- the built-in preprocessor tokenizes them on the fly.
history = classifier.fit(
    x=train_texts,
    y=train_labels,
    validation_data=(val_texts, val_labels),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    callbacks=[early_stop],
)
  
```

...

Epoch 1/3	263/263	204s	529ms/step	- accuracy: 0.8879	- loss: 0.4066	- val_accuracy: 0.9422	- val_loss: 0.2082
Epoch 2/3	263/263	107s	405ms/step	- accuracy: 0.9569	- loss: 0.1736	- val_accuracy: 0.9511	- val_loss: 0.1896
Epoch 3/3	263/263	107s	405ms/step	- accuracy: 0.9760	- loss: 0.1038	- val_accuracy: 0.9511	- val_loss: 0.1809

9) Loss and Accuracy over Epochs



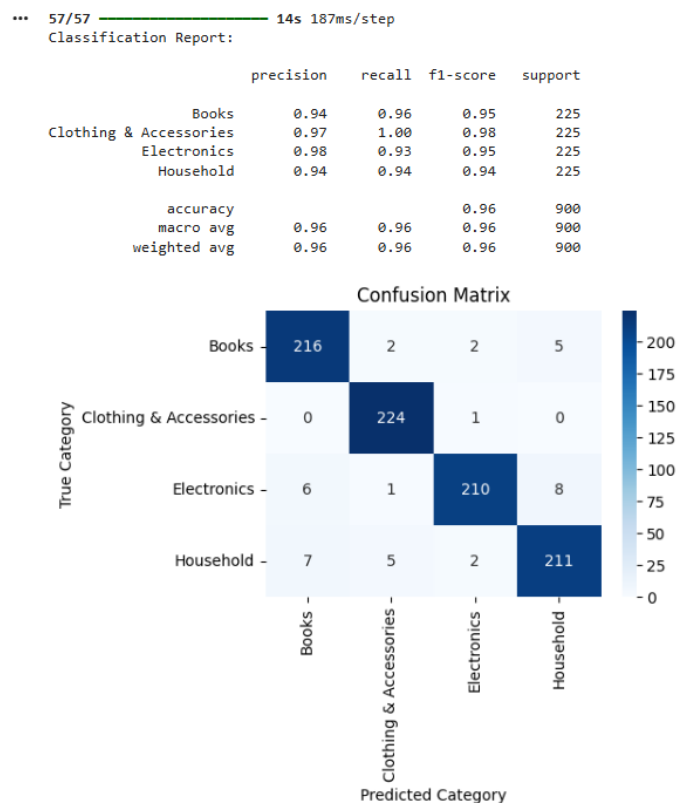
10) Evaluating model on test set

```
# Evaluate overall loss & accuracy on the held-out test set
test_loss, test_accuracy = classifier.evaluate(x=test_texts, y=test_labels, batch_size=BATCH_SIZE)
print(f"\nTest Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")
```

57/57 ————— 7s 121ms/step - accuracy: 0.9567 - loss: 0.1795

Test Loss: 0.1795
Test Accuracy: 0.9567

11) Classification report and confusion matrix



12) Evaluating the model on custom sample text

```
def predict_category(text, classifier=classifier, label_encoder=label_encoder):
    """Cleans the text and predicts the category using the fine-tuned classifier."""
    cleaned = clean_text(text)
    probs = classifier.predict(x=[cleaned], batch_size=1, verbose=0)[0]
    pred_label = np.argmax(probs)
    pred_category = label_encoder.inverse_transform([pred_label])[0]
    confidence = probs[pred_label]
    return pred_category, confidence

# Try a few sample product descriptions
sample_texts = [
    "Samsung 32 inch smart LED TV with HDR support and built-in WiFi",
    "Cotton casual t-shirt for men, slim fit, available in multiple colors",
    "The Alchemist a novel by Paulo Coelho, paperback edition",
    "Stainless steel kitchen knife set with wooden storage block",
]

for text in sample_texts:
    category, confidence = predict_category(text)
    print(f"Text: {text}")
    print(f" -> Predicted Category: {category} (confidence: {confidence:.2f})\n")
```

```
*** Text: Samsung 32 inch smart LED TV with HDR support and built-in WiFi
    -> Predicted Category: Electronics (confidence: 0.82)

Text: Cotton casual t-shirt for men, slim fit, available in multiple colors
    -> Predicted Category: Clothing & Accessories (confidence: 1.00)

Text: The Alchemist a novel by Paulo Coelho, paperback edition
    -> Predicted Category: Books (confidence: 0.99)

Text: Stainless steel kitchen knife set with wooden storage block
    -> Predicted Category: Household (confidence: 0.99)
```

Conclusion :

- We fine-tuned a pretrained BERT model (bert_base_en_uncased) using TensorFlow/Keras (via KerasHub's BertTextClassifier) to classify e-commerce product descriptions into their product category.
- The pipeline covered: downloading data with kagglehub, cleaning text, loading a ready-to-train pretrained BERT classifier (tokenization included), training, and evaluating the model.
- The model achieves strong accuracy even with a small subsample of data and only a few epochs, demonstrating the power of transfer learning with pretrained transformer models.
- Possible improvements: use the full dataset (SAMPLE_SIZE = None), train for more epochs, increase MAX_LEN, try a larger preset (e.g. bert_large_en_uncased), or add learning-rate scheduling / warmup.